

21l-5692

March 12, 2024

1 Muhammad Hassan Khalid

2 21L-5692

2.1 Question 1

```
[ ]: import numpy as np
import pandas as pd
from itertools import combinations
import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
```

The problem is to match the user's free-form input against a pre-determined list of banks. For example, user input 'bawag bank' should be matched to 'BAWAG Group AG'.

```
[2]: import numpy as np
import pandas as pd
from itertools import combinations
from difflib import SequenceMatcher
import re
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize

# List of banks to compare
banks = ['Sberbank Europe AG', 'BAWAG Group AG', 'Raiffeisenbankengruppe OÖ',
↳ 'Verbund eGen',
↳ 'Raiffeisen Bank International AG', 'Volksbanken Verbund', 'Erste',
↳ 'Group Bank AG', 'KBC Groep',
↳ 'Investeringsmaatschappij Argenta', 'Belfius Bank', 'AXA Bank Belgium',
↳ 'The Bank of New York Mellon SA/NV', 'First Investment Bank AD', 'RCB',
↳ 'Bank Ltd',
↳ 'Bank of Cyprus Holdings Public Limited Company', 'Hellenic Bank',
↳ 'Public Company Limited',
↳ 'DekaBank Deutsche Girozentrale', 'Erwerbsgesellschaft der',
↳ 'S-Finanzgruppe mbH & Co. KG',
```

'UBS Europe SE', 'DEUTSCHE APOTHEKER- UND ÄRZTEBANK EG',
 'Volkswagen Bank Gesellschaft mit beschränkter Haftung', 'Münchener
 ↪Hypotheekenbank eG',
 'DZ BANK AG Deutsche Zentral-Genossenschaftsbank, Frankfurt am Main',
 ↪'HASPA Finanzholding',
 'State Street Europe Holdings Germany S.a.r.l. & Co. KG', 'J.P. Morgan
 ↪AG',
 'DEUTSCHE BANK AKTIENGESELLSCHAFT', 'COMMERZBANK Aktiengesellschaft',
 ↪'Landesbank Baden-Württemberg',
 'Landesbank Hessen-Thüringen Girozentrale', 'Norddeutsche Landesbank -
 ↪Girozentrale -',
 'Deutsche Pfandbriefbank AG', 'Aareal Bank AG', 'Hamburg Commercial
 ↪Bank AG', 'Bayerische Landesbank',
 'Jyske Bank A/S', 'Sydbank A/S', 'Nykredit Realkredit A/S', 'Danske
 ↪Bank A/S', 'Luminor Holding AS',
 'Abanca Corporacion Bancaria S.A.', 'Banco Santander S.A.', 'Ibercaja
 ↪Banco S.A.', 'Kutxabank S.A.',
 'Unicaja Banco S.A.', 'CaixaBank S.A.', 'Banco de Crédito Social
 ↪Cooperativo',
 'Banco Bilbao Vizcaya Argentaria S.A.', 'Banco de Sabadell S.A.',
 ↪'Bankinter S.A.', 'Kuntarahoitus Oyj',
 'Nordea Bank Abp', 'OP Osuuskunta', 'SFIL', 'RCI Banque',
 ↪'Confédération Nationale du Crédit Mutuel',
 'La Banque Postale', 'Bpifrance', 'C.R.H. - Caisse de refinancement de
 ↪l'habitat',
 'HSBC Continental Europe', 'Groupe BPCE', 'Groupe Crédit Agricole',
 ↪'Société générale', 'BNP Paribas',
 'ALPHA SERVICES AND HOLDINGS S.A.', 'National Bank of Greece S.A.',
 'Eurobank Ergasias Services and Holdings S.A.', 'Piraeus Financial
 ↪Holdings', 'OTP-csoport',
 'Magyar Bankholding', 'Barclays Bank Ireland plc', 'Citibank Holdings
 ↪Ireland Limited', 'AIB Group plc',
 'Bank of Ireland Group plc', 'Ulster Bank Ireland Designated Activity
 ↪Company',
 'Bank of America Europe Designated Activity Company', 'Íslandsbanki hf.
 ↪', 'Landsbankinn hf.',
 'Arion banki hf', 'Intesa Sanpaolo S.p.A.', 'Gruppo Bancario
 ↪Finecobank', 'UniCredit S.p.A.',
 'Gruppo Bancario Mediolanum', 'Credito Emiliano Holding S.p.A.',
 ↪'Banco BPM SpA',
 'Banca Popolare di Sondrio, Società Cooperativa per Azioni', 'Banca
 ↪Monte dei Paschi di Siena S.p.A.',
 'CASSA CENTRALE BANCA', 'ICCREA BANCA S.P.A.', 'Mediobanca - Banca di
 ↪Credito Finanziario S.p.A.',

```

'Akcinė bendrovė Šiaulių bankas', 'Precision Capital S.A.', 'RBC
↳Investor Services Bank S.A.',
'J.P. Morgan Bank Luxembourg S.A.', 'Banque Internationale à
↳Luxembourg',
'Banque et Caisse d'Épargne de l'État, Luxembourg', 'Akcijsabiedrība
↳"Citadele banka"',
'MDB Group Limited', 'Bank of Valletta Plc', 'HSBC Bank Malta p.l.c.',
↳'BNG Bank N.V.', 'ING Groep N.V.',
'LP Group B.V.', 'de Volksbank N.V.', 'ABN AMRO Bank N.V.',
↳'Coöperatieve Rabobank U.A.',
'Nederlandse Waterschapsbank N.V.', 'Bank Polska Kasa Opieki S.A.',
'Powszechna Kasa Oszczędności Bank Polski S.A.', 'LSF Nani Investments
↳S.à r.l.',
'Banco Comercial Português SA', 'Caixa Geral de Depósitos SA', 'Banca
↳Transilvania',
'Lämförsäkringsbank AB (publ)', 'Kommuninvest - group',
↳'Skandinaviska Enskilda Banken - group',
'SBAB Bank AB - group', 'Swedbank - group', 'Svenska Handelsbanken -
↳group', 'Biser Topco S.à r.l.',
'Nova Ljubljanska Banka d.d. Ljubljana']

```

Examples of search strings

```
s1 = 'Bawag bank' # other options: 'Bawag bank', 'Erste', 'Raiffeisen bank'
```

Preprocessing functions

```

def preprocess_text(text):
    # Case normalization
    text = text.lower()

    # Punctuation removal
    text = re.sub(r'[\W\s]', '', text)

    # Stop word removal
    stop_words = set(stopwords.words('english'))
    word_tokens = word_tokenize(text)
    filtered_text = [word for word in word_tokens if word not in stop_words]
    text = ' '.join(filtered_text)

    # Stemming
    stemmer = PorterStemmer()
    text = ' '.join([stemmer.stem(word) for word in word_tokenize(text)])

    # Lemmatization
    lemmatizer = WordNetLemmatizer()
    text = ' '.join([lemmatizer.lemmatize(word) for word in
↳word_tokenize(text)])

```

```

    return text

# Preprocess user query
s1_processed = preprocess_text(s1)

# Preprocess bank names
banks_processed = [preprocess_text(bank) for bank in banks]

# Similarity calculation
res = []
for token in banks_processed:
    res.append([s1_processed, token, SequenceMatcher(None, s1_processed, token).
        ↪ratio()])

df2 = pd.DataFrame(res, columns=['Bank 1', 'Bank 2', 'Score'])
# Sort by score
df2_sorted = df2.sort_values(by=['Score'], ascending=[False])

# Output
print(df2_sorted.head())

```

	Bank 1	Bank 2	Score
99	bawag bank	bng bank nv	0.666667
37	bawag bank	dansk bank	0.600000
1	bawag bank	bawag group ag	0.583333
31	bawag bank	aareal bank ag	0.583333
8	bawag bank	belfiu bank	0.571429

3 Calculated using Jaccard similarity

```

[3]: def calculate_similarity(str1, str2):
    # Convert strings to lowercase
    str1_lower = str1.lower()
    str2_lower = str2.lower()

    # Tokenize strings
    tokens1 = str1_lower.split()
    tokens2 = str2_lower.split()

    # Count the number of matching words
    match_count = sum(word in str2_lower for word in tokens1) + sum(word in_
        ↪str1_lower for word in tokens2)

    # Calculate Jaccard similarity
    total_words = len(tokens1) + len(tokens2)

```

```

        similarity = match_count / total_words if total_words != 0 else 0

    return similarity

# Example of search strings
s1 = 'Bawag bank' # other options: 'Bawag bank', 'Erste', 'Raiffeisen bank'

# Initialize list to store similarity scores
scores = []

# Iterate through bank names and calculate similarity scores
for bank in banks:
    score = calculate_similarity(s1, bank)
    scores.append((bank, score))

# Sort the scores in descending order
scores.sort(key=lambda x: x[1], reverse=True)

# Display the top matches
top_matches = scores[:5] # Adjust the number of top matches as needed
for match in top_matches:
    print("Bank: {}, Score: {}".format(match[0], match[1]))

```

```

Bank: BAWAG Group AG, Score: 0.6
Bank: Aareal Bank AG, Score: 0.6
Bank: Raiffeisen Bank International AG, Score: 0.5
Bank: Erste Group Bank AG, Score: 0.5
Bank: Belfius Bank, Score: 0.5

```

3.1 Question 2

```

[ ]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression, Perceptron
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load the dataset with specified encoding
dataset_path = "Q2 Sentiment Analysis Dataset.csv"
data = pd.read_csv(dataset_path, encoding='ISO-8859-1')

# Preprocess the text data
def preprocess_text(text):

```

```

# Convert text to lowercase
text = text.lower()
# Remove punctuation
text = ''.join([c for c in text if c not in '!"#$%&\'()*+,-./:;<=>?
↪@[\]\^_`{|}~'])
return text

data['text_processed'] = data['text'].apply(preprocess_text)

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(data['text_processed'],
↪data['sentiment'], test_size=0.2, random_state=42)

# Define classifiers
classifiers = {
    'Naive Bayes': MultinomialNB(),
    'Logistic Regression': LogisticRegression(),
    'Random Forest': RandomForestClassifier(),
    'SVM': SVC(),
    'Perceptron': Perceptron()
}

# Define feature extraction methods
vectorizers = {
    'Bag of Words (Raw Counts)': CountVectorizer(),
    'Bag of Words (TF-IDF)': TfidfVectorizer(),
    'N-grams (unigrams, bigrams, trigrams)': CountVectorizer(ngram_range=(1, 3))
}

# Train and evaluate classifiers for each feature extraction method
results = {}
for vec_name, vectorizer in vectorizers.items():
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)

    results[vec_name] = {}
    for clf_name, clf in classifiers.items():
        clf.fit(X_train_vec, y_train)
        y_pred = clf.predict(X_test_vec)

        acc = accuracy_score(y_test, y_pred)
        precision = precision_score(y_test, y_pred, average='macro')
        recall = recall_score(y_test, y_pred, average='macro')
        f1 = f1_score(y_test, y_pred, average='macro')

        results[vec_name][clf_name] = {'Accuracy': acc, 'Precision': precision,
↪'Recall': recall, 'F1-score': f1}

```

```
[15]: from tabulate import tabulate

# Display results
print("Results:")
for vec_name, vec_results in results.items():
    print("\nFeature Extraction Method:", vec_name)
    table_data = []
    headers = ['Classifier', 'Accuracy', 'Precision', 'Recall', 'F1-score']
    for clf_name, metrics in vec_results.items():
        acc = metrics['Accuracy']
        precision = metrics['Precision']
        recall = metrics['Recall']
        f1 = metrics['F1-score']
        table_data.append([clf_name, acc, precision, recall, f1])
    print(tabulate(table_data, headers=headers, tablefmt='grid'))
```

Results:

Feature Extraction Method: Bag of Words (Raw Counts)

Classifier	Accuracy	Precision	Recall	F1-score
Naive Bayes	0.715938	0.496166	0.428204	0.424428
Logistic Regression	0.757069	0.518944	0.458797	0.470749
Random Forest	0.746787	0.532731	0.44838	0.462015
SVM	0.742931	0.600598	0.425956	0.435329
Perceptron	0.700514	0.508089	0.481661	0.489879

Feature Extraction Method: Bag of Words (TF-IDF)

Classifier	Accuracy	Precision	Recall	F1-score
Naive Bayes	0.748072	0.627505	0.421285	0.419845
Logistic Regression	0.751928	0.526051	0.441169	0.449938
Random Forest	0.748072	0.578536	0.443283	0.457808
SVM	0.75964	0.610379	0.44147	0.451787
Perceptron	0.730077	0.524202	0.498389	0.507829

Feature Extraction Method: N-grams (unigrams, bigrams, trigrams)

Classifier	Accuracy	Precision	Recall	F1-score
Naive Bayes	0.737789	0.525829	0.437771	0.43632
Logistic Regression	0.748072	0.526549	0.446487	0.459279
Random Forest	0.737789	0.566084	0.416788	0.42609
SVM	0.703085	0.580548	0.390556	0.397322
Perceptron	0.726221	0.511729	0.450486	0.466855

```
[16]: # Initialize results table
results_table = []

# Calculate micro and macro averages
micro_avg = {}
macro_avg = {}
for metric in ['Accuracy', 'Precision', 'Recall', 'F1-score']:
    micro_avg[metric] = sum(results[vec_name][clf_name][metric] for vec_name in
    ↪ results for clf_name in results[vec_name]) / len(data)
    macro_avg[metric] = sum(results[vec_name][clf_name][metric] for vec_name in
    ↪ results for clf_name in results[vec_name]) / len(results) / len(classifiers)

# Append micro and macro averages to the results table
results_table.append(['Micro Average', micro_avg['Accuracy'],
    ↪ micro_avg['Precision'], micro_avg['Recall'], micro_avg['F1-score']])
results_table.append(['Macro Average', macro_avg['Accuracy'],
    ↪ macro_avg['Precision'], macro_avg['Recall'], macro_avg['F1-score']])

# Append classifier results to the results table
for clf_name in classifiers:
    clf_results = [clf_name]
    for metric in ['Accuracy', 'Precision', 'Recall', 'F1-score']:
        clf_results.append(sum(results[vec_name][clf_name][metric] for vec_name
        ↪ in results) / len(results))
    results_table.append(clf_results)

# Display results table
headers = ['Classifier', 'Accuracy', 'Precision', 'Recall', 'F1-score']
print(tabulate(results_table, headers=headers, tablefmt='grid'))
```

+-----+-----+-----+-----+-----+

Classifier	Accuracy	Precision	Recall	F1-score
Micro Average	0.00284457	0.00211887	0.0017063	0.00173841
Macro Average	0.736932	0.548929	0.442045	0.450365
Naive Bayes	0.733933	0.549834	0.429087	0.426864
Logistic Regression	0.752356	0.523848	0.448817	0.459989
Random Forest	0.744216	0.559117	0.43615	0.448638
SVM	0.735219	0.597175	0.419327	0.428146
Perceptron	0.718937	0.514673	0.476846	0.488188

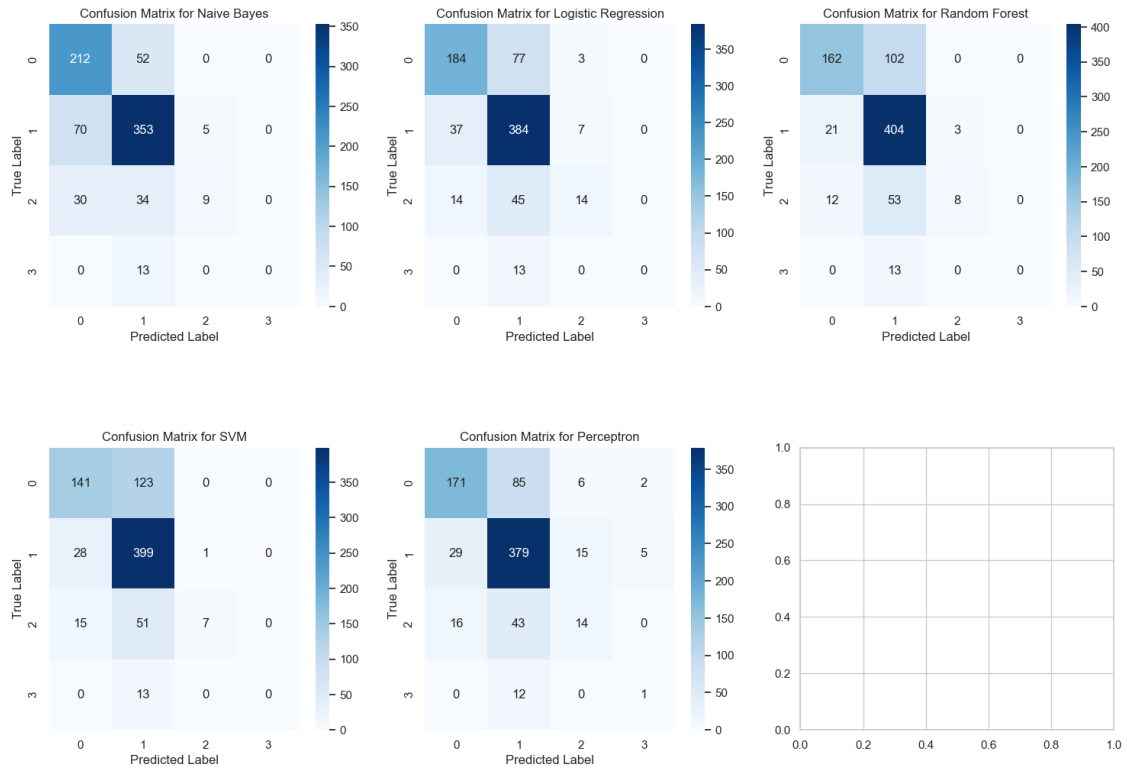
```
[20]: import seaborn as sns
import matplotlib.pyplot as plt

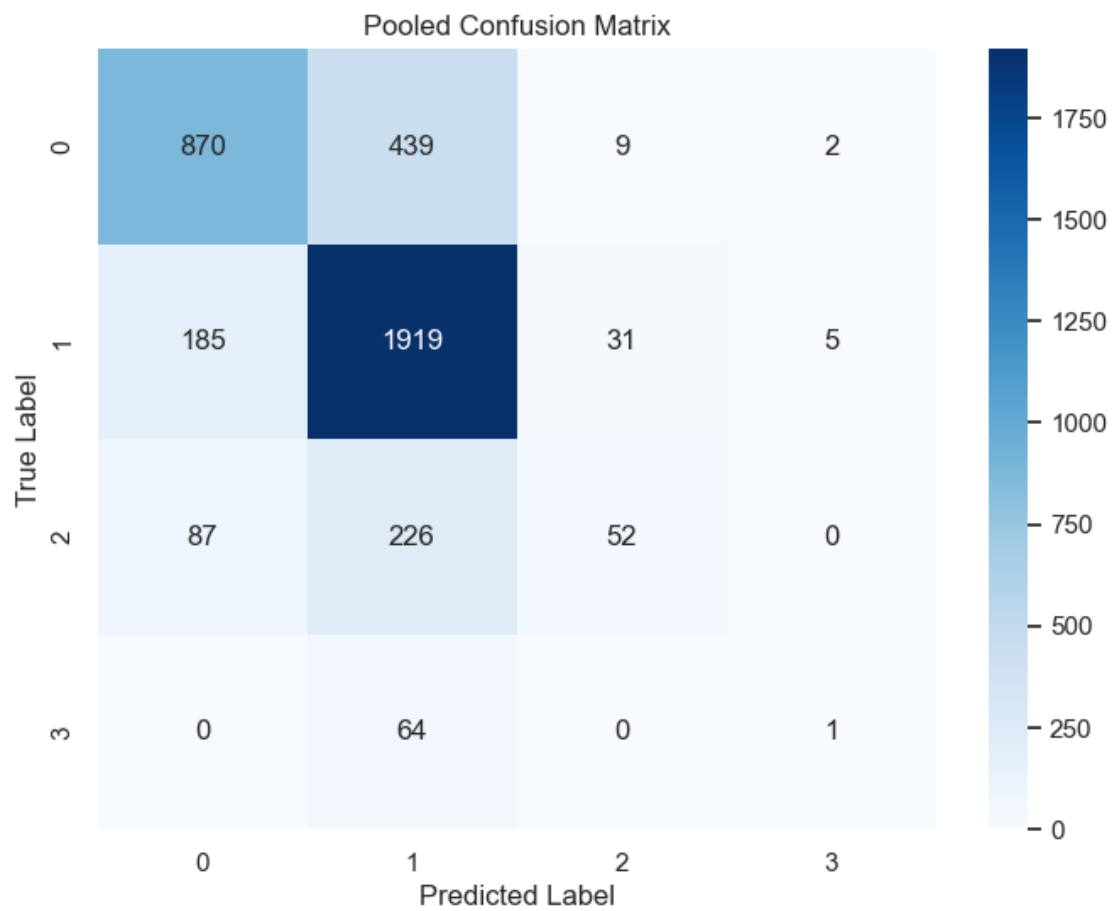
# Set seaborn style
sns.set(style="whitegrid")

# Create a figure with subplots for individual confusion matrices
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(18, 12))
plt.subplots_adjust(hspace=0.5)

# Plot individual confusion matrices
for i, (clf_name, conf_matrix) in enumerate(conf_matrices.items()):
    ax = axes[i//3, i%3]
    sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", ax=ax)
    ax.set_title(f"Confusion Matrix for {clf_name}")
    ax.set_xlabel("Predicted Label")
    ax.set_ylabel("True Label")

# Plot pooled confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(pooled_conf_matrix, annot=True, fmt="d", cmap="Blues")
plt.title("Pooled Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```





[]: